

Department of Computer Science and Engineering
CSE 422: Computer Graphics Lab

Session 2: Perform to design output primitives using OpenGL

Intended Learning Outcome:

- a. Able to demonstrate effective OpenGL programs to solve graphics programming issues including different shapes.
- b. Able to appreciate the knowledge along axis (X,Y).

Expected Skills:

Gain necessary knowledge to make some designs using OpenGL.

Tools Required:

- a. CodeBlocks
- b. OpenGL and GLUT using CodeBlocks.

Session Detail:

Department of Computer Science and Engineering
CSE 422: Computer Graphics Lab

```
#include <GL/gl.h>
#include <GL/glut.h>

/* everything inside the function display() appears on the
screen. */

void display(void)
{
/* glClear() clears the screen using the background color
specified earlier */
/* GL_COLOR_BUFFER_BIT means clear the color buffer
(screen) */
glClear (GL_COLOR_BUFFER_BIT);

/* basically erase all previously drawn colors and start
fresh. */

///1st body
glColor3f (1.0, 1.0, 1.0); /* glColor3f(R, G, B) */

// glColor3f(1,0,0); // red
// glColor3f(0,1,0); // green
// glColor3f(0,0,1); // blue
// glColor3f(1,1,1); // white
// glColor3f(0,0,0); // black

glBegin(GL_POLYGON);
/* OpenGL connects all vertices in order and
fills the shape */

glVertex3f (0.53, 0.42, 0.0);
glVertex3f (0.56, 0.34, 0.0);
glVertex3f (0.59, 0.42, 0.0);
glVertex3f (0.67, 0.45, 0.0);
glVertex3f (0.59, 0.48, 0.0);
glVertex3f (0.56, 0.56, 0.0);
glVertex3f (0.53, 0.48, 0.0);
glVertex3f (0.45, 0.45, 0.0);
glEnd();

///2nd body
glColor3f (1.0, 0.0, 0.0);
glBegin(GL_POLYGON);
glVertex3f (0.31, 0.42, 0.0);
glVertex3f (0.34, 0.34, 0.0);
glVertex3f (0.37, 0.42, 0.0);
```

Department of Computer Science and Engineering
CSE 422: Computer Graphics Lab

```
glVertex3f (0.45, 0.45, 0.0);
glVertex3f (0.37, 0.48, 0.0);

glVertex3f (0.34, 0.56, 0.0);
glVertex3f (0.31, 0.48, 0.0);
glVertex3f (0.23, 0.45, 0.0);
glEnd();

///2nd body
glColor3f (0.0, 1.0, 0.0);
glBegin(GL_POLYGON);
glVertex3f (0.42, 0.53, 0.0);
glVertex3f (0.45, 0.45, 0.0);
glVertex3f (0.48, 0.53, 0.0);
glVertex3f (0.56, 0.56, 0.0);
glVertex3f (0.48, 0.59, 0.0);
glVertex3f (0.45, 0.67, 0.0);
glVertex3f (0.42, 0.59, 0.0);
glVertex3f (0.34, 0.56, 0.0);
glEnd();

///2nd body
glColor3f (0.0, 0.0, 1.0);
glBegin(GL_POLYGON);
glVertex3f (0.42, 0.31, 0.0);
glVertex3f (0.45, 0.23, 0.0);
glVertex3f (0.48, 0.31, 0.0);
glVertex3f (0.56, 0.34, 0.0);
glVertex3f (0.48, 0.37, 0.0);
glVertex3f (0.45, 0.45, 0.0);
glVertex3f (0.42, 0.37, 0.0);
glVertex3f (0.34, 0.34, 0.0);
glEnd();

/* start processing buffered OpenGL routines */
glFlush ();
}
```

Department of Computer Science and Engineering
CSE 422: Computer Graphics Lab

```
void init (void)
{
    /* clearing (background) color / sets it to black */
    glClearColor (0.0, 0.0, 0.0, 0.0);

    /* initialize viewing values or configures the camera */
    glMatrixMode(GL_PROJECTION);

    glLoadIdentity(); //resets any previous transformation
    glOrtho(0.0, 1.0, 0.0, 1.0,-1.0, 1.0);
    // creates 2D coordinate space
    /* This treats the window as a graph */
}

/*
*Declare initial window size, position, and display mode
*(single buffer and RGBA). Open window with "hello"
*in its title bar. Call initialization routines.
*Register callback function to display graphics.
*Enter main loop and process events.
*/

int main(int argc, char** argv)
{
    glutInit(&argc, argv); // glut initialization

    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    // single buffering and RGB colors

    glutInitWindowSize (500, 500);
    // size 500x500

    glutInitWindowPosition (100, 100);
    // 100 pixels from left
    // 100 pixels from top

    glutCreateWindow ("Star Designing");
    init ();
    glutDisplayFunc(display);
    glutMainLoop();

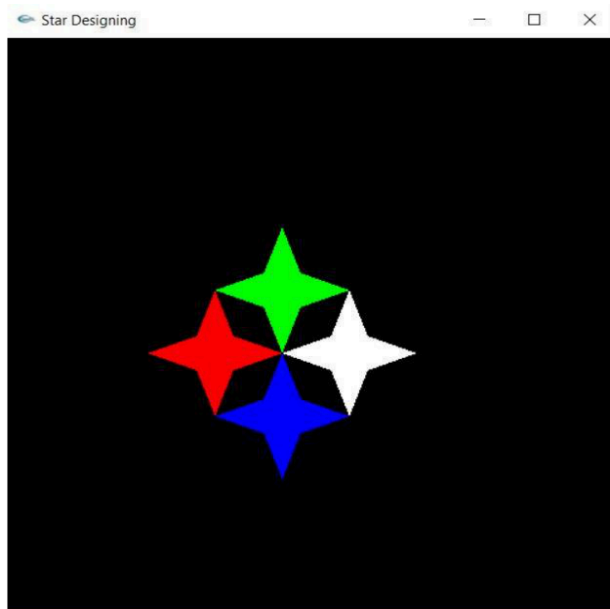
    /* without this line the window would immediately close
    */
    return 0; /* ISO C requires main to return int. */
}
```

Department of Computer Science and Engineering
CSE 422: Computer Graphics Lab

Sample Input:

No sample input is needed.

Sample Output:



Daffodil
**International
University**

